

Modern Cryptography – Part 1

Stream Ciphers & Block Ciphers

1. Block Cipher（分组密码）

将一整块明文作为一个整体进行加密，并产生一个相同长度的密文块（通常为64位或128位）

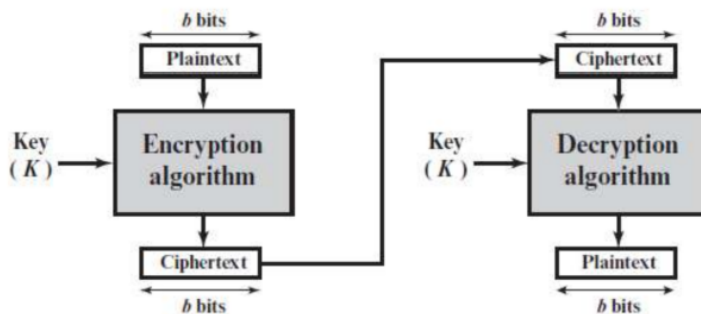
将明文分成固定大小的数据块，每次加密一个数据块。

类比：数据被装进一个个箱子（Block），然后整箱运输（加密）。

Algorithm	Block Size
DES	64 bits
AES	128 bits

加密前后，plaintext 和 ciphertext 长度一致

- 加密 Encryption: $C = E_K(P)$
- 解密 Decryption: $P = D_K(C)$
- Block of plaintext is treated as a whole and used to produce a ciphertext block of equal length (64 / 128 bits).



1.1. Block Cipher Characteristics

- Variable (or flexible) of
 - key length / block size / no. of rounds
- Mixed operators, data
- Key dependent rotation
- Key dependent S-boxes
- More complex key scheduling (Variable F)
- Operation of full data in each round
- Varying non-linear functions

1. **Variable** (or flexible) of key length / block size / no. of rounds 密钥长度、分组长度、轮数

- Key length: Key 越长, 攻击难度越大

算法	密钥长度
DES	56 bits
AES-128	128 bits
AES-192	192 bits
AES-256	256 bits

例如, 暴力破解56位: 2^{56} vs 256位: 2^{256} 差距极其巨大。

- Block Size: 分组越大, 模式攻击更困难, 重复块出现概率更低
 - DES: 64 bits
 - AES: 128 bits
- Number of Rounds: 加密步骤重复执行多少次 -> 多次变换更加复杂
 - AES-128: 10 Rounds
 - AES-192: 12 Rounds
 - AES-256: 14 Rounds

2. Mixed operators, data

现代密码不会只用一种运算。

而是同时使用:

- XOR: $A \oplus B$

- AND: $A \wedge B$
- OR: $A \vee B$
- Addition: $(A + B) \bmod 2$
- Rotation 循环移位: 101101 –左移2位→110110

如果只用 XOR: 规律太容易分析

多种运算混合后: 数学结构更复杂, 更难攻击。

3. Key dependent rotation 依赖密钥的循环移位: 移多少位由密钥决定

- 例如, 密钥最后3位: 101, 即 5 --> 左移 5 位

4. Key dependent S-boxes 依赖密钥的S盒

- S-box: Substitution Box 替换 (类似于字典, 有一个查表规则)
- 本质: 输入一个值, 输出另一个值

5. More complex key scheduling (Variable F) 更加复杂的密钥扩展机制

- Key Schedule 主密钥 K 并不会直接用于每一轮, 而是通过 Key schedule 生成每轮子密钥
- Variable F, Round Function 轮函数: 表示每轮使用不同规则。

6. Operation of full data in each round 每轮处理全部数据

- 早期密码: 每轮可能只处理部分数据
- 现代密码: 每轮都尽可能影响整个数据块。

7. Varying non-linear functions

- 如果全是线性运算: 攻击者可以用矩阵运算、
- 非线性, 规则更复杂

Variable key length	可变密钥长度	提高暴力破解难度
Variable block size	可变分组长度	减少模式暴露
Variable rounds	可变轮数	增强混淆扩散
Mixed operators	混合运算	增加复杂性
Key dependent rotation	密钥控制移位	增加随机性
Key dependent S-box	密钥控制S盒	提高抗分析能力
Complex key scheduling	复杂密钥扩展	生成不同轮密钥
Full data operation	每轮处理整个分组	快速扩散

Non-linear functions	非线性函数	防止数学破解
----------------------	-------	--------

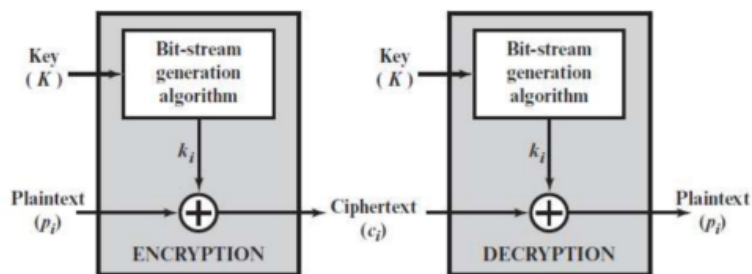
2. Stream Cipher (流密码)

流密码每次对一个比特(bit)或一个字节(byte)的数据流进行加密。

类比：数据像流水一样连续通过加密器。

- 加密： $c_i = p_i \oplus k_i$
 - p_i = 第 i 位明文
 - k_i = 密钥流 (Keystream)
 - c_i = 第 i 位密文
 - $\oplus$ = XOR (异或)
- 解密： $p_i = c_i \oplus k_i$
 - 因为， $(p_i \oplus k_i) \oplus k_i = p_i$ 利用异或运算的性质即可恢复原文
 - 加密：明文 $\oplus$ 密钥 = 密文
 - 解密：密文 $\oplus$ 密钥 = 明文

- Encrypts a digital data stream **one bit or one byte** at a time.
- classical stream ciphers are the autokeyed Vigenère cipher and the Vernam cipher.



常见流密码：

- Autokeyed Vigenère Cipher 自动密钥维吉尼亚密码 – 一种古典流密码
- Vernam Cipher 维尔南密码

▼ 【拓展】

1. Autokeyed Vigenère Cipher

普通维吉尼亚密码

① 密钥重复拓展

- 明文: H E L L O
- 密钥: K E Y K E

② 字母转换数字并累加

Autokey (自动密钥) 思想

普通维吉尼亚有一个问题: 密钥会重复, 攻击者可以利用重复规律破解。

于是, Autokey Vigenère, 先使用关键字: KEY

之后把明文接到密钥后面: KEYHE, 直到密钥长度等于明文长度, 作为整个密钥流。

然后, 一一对应相加!

2. Vernam Cipher

Gilbert Vernam 1917年提出。

核心思想: 用 XOR 异或 进行加密, $C = P \oplus K$

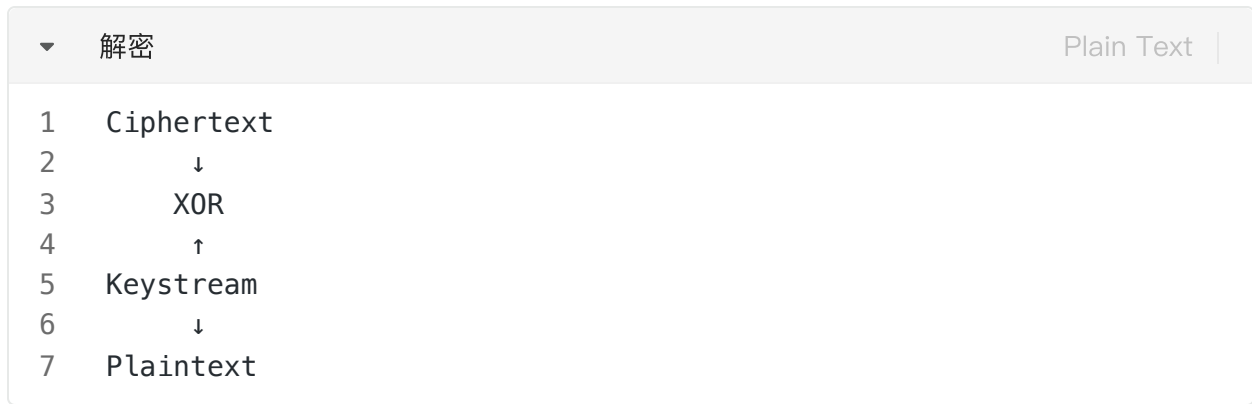
为什么能解密? (我称之为叹为观止的聪明)

- 因为 $A \oplus A = 0$; $A \oplus 0 = A$
- 因此, $(P \oplus K) \oplus K = P \oplus (K \oplus K) = P \oplus 0 = P$

因为 XOR 本质就是: 模2加法 (Addition Mod 2)

- 异或可以写成: $A \oplus B = (A + B) \bmod 2$
- 普通加法有结合律, 因此模2加法也有结合律

▼	加密	Plain Text
1	Plaintext	
2	↓	
3	XOR	
4	↑	
5	Keystream	
6	↓	
7	Ciphertext	



注意：加密和解密完全一样

Vernam Cipher进一步发展后，如果满足：

- 条件1：密钥完全随机(Random)
- 条件2：密钥长度 = 明文长度
- 条件3：密钥只使用一次 (One Time)

则得到：One-Time Pad 一次一密系统 -> Perfect secrecy 理论上不可破解

3. 对比

项目	Block Cipher 分组密码	Stream Cipher 流密码
加密单位	一整块数据	一个bit/byte
工作方式	Block-by-Block	Continuous Stream
是否需要Padding	需要	不需要
速度	较慢	较快
错误传播	容易扩散	影响较小
典型算法	AES、DES	RC4、Vernam
应用场景	文件加密、数据库	视频、语音、WiFi

DES Encrytion Standard

● Data Encryption Standard (DES):

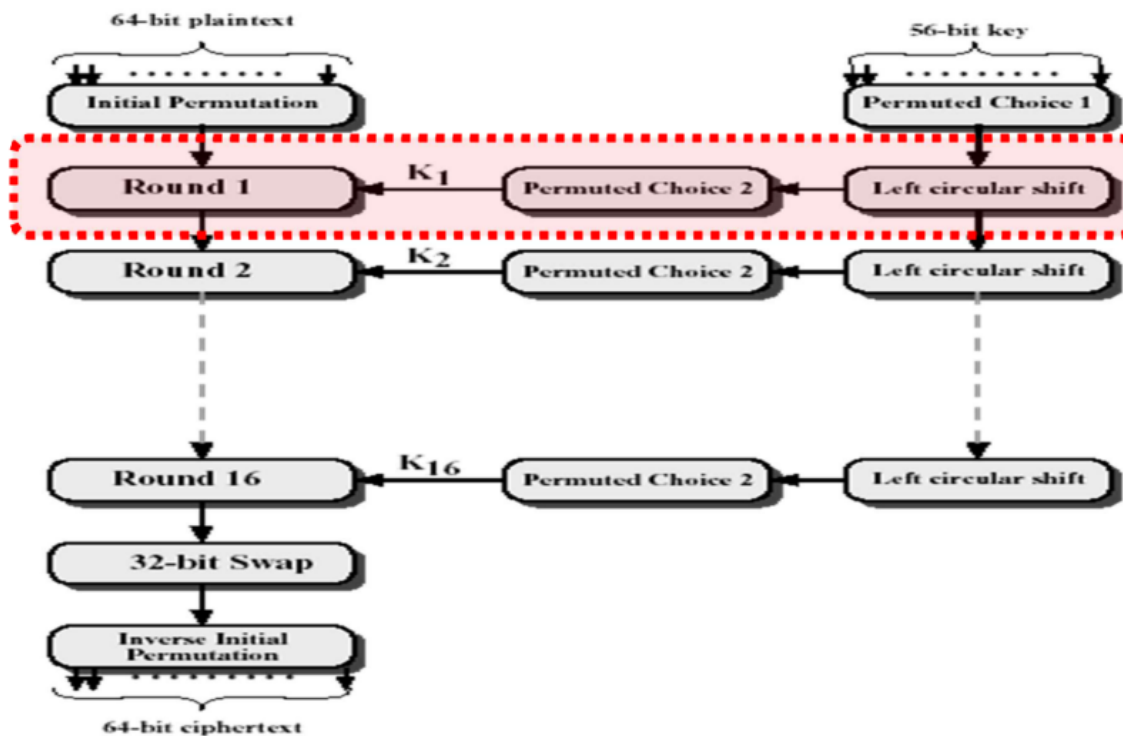
- A **symmetric-key** algorithm for the encryption of digital data.
- Although its short key length of 56 bits makes it too insecure for applications,
- it has been highly influential in the advancement of cryptography.
- **Key size – 56 bits; Block size – 64 bits**

■ History

- Developed in the early 1970s at IBM.
- Submitted to the National Bureau of Standards (NBS).
- In 1976, after consultation with the National Security Agency (NSA).

1. Symmetric-key (对称密钥)：加密和解密使用同一个密钥
2. 尽管，对于目前的技术来说，56 bits 的 key 是 insecure 且可以被暴力破解的，但是也对密码学的发展产生了巨大的影响
3. Key size 密钥大小 vs Block size 每次处理数据的大小

DES Algorithm



- 左边 数据流

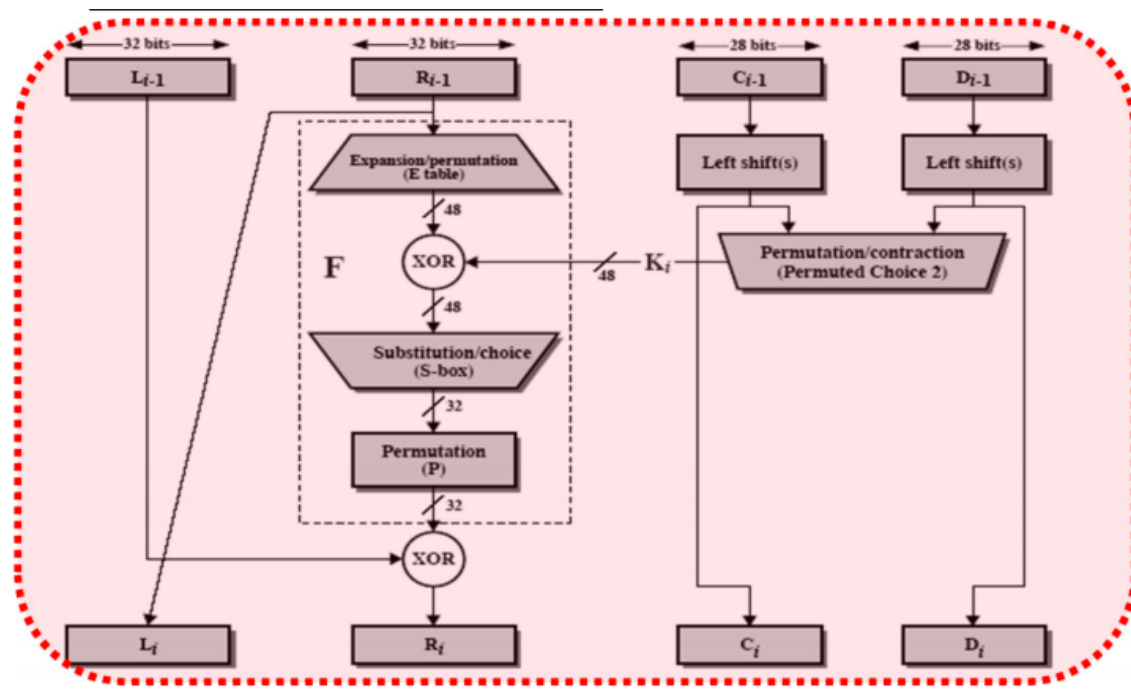
- 右边 密钥生成过程

1. Initial Permutation (IP)

- 初始置换：重新排列 64 位数据 -> 只是换位置

2. 16 轮加密

- 右边：子密钥生成（生成 $K_1, K_2, \dots, K_{16}$ ）



- DES采用：Feistel Network 费斯妥结构

- 核心结构：

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

数据流变化（左半边）

Step 1: 数据分两半

上一轮数据：64 bits

分成：

- $L(i-1) = 32$ bits
- $R(i-1) = 32$ bits

Step 2: 左边直接复制

$L_i = R(i-1)$ 即，新的左半边 = 旧的右半边

Step 3: 右边进入 F 函数

图中虚线框是 DES 安全性的核心

1. F 函数第一步 Expansion Permutation (E)：扩展，即 32 bits $\rightarrow$ 48 bits

- E-box 的作用：把 32位 拓展为 48 位
- 2. XOR : $R \oplus K_i$
- 3. S-box Substitution Box: 引入 Nonlinearity, 48 bits $\rightarrow$ 32 bits
- DES 的非线性来自哪里? Ans: S-Box
- 4. Permutation (P) : 再次打乱顺序, 作用: Diffusion 扩散, 32 bits $\rightarrow$ 32 bits

Step 4: 与左半边异或

产生新的右半边: $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$

子密钥生成 (右半图)

- DES 密钥长度: 56位是真正密钥 (真正参与加密) + 8位是Parity Bit (奇偶校验位)
- 图中: 循环左移 (为了让每轮使用的密钥不同) $\rightarrow$ Permuted Choice 2 (PC -2) $\rightarrow K_i$
 - 去掉 8 个校验位 64 bits $\rightarrow$ 56 bits
 - 重新排列顺序

Digital Logic

● Digital Electronics:

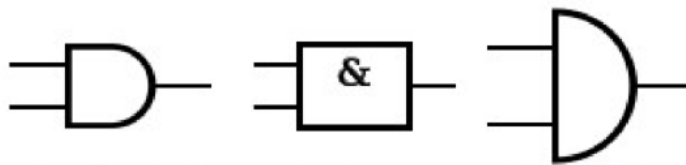
- Field of electronics involving the study of digital signals and the engineering of devices that use or produce them.
- This is in contrast to analog electronics and analog signals.
- Complex devices may have simple electronic representations of Boolean logic functions.

● Boolean Logic (Boolean Algebra)

- The branch of algebra in which the values of the variables are the truth values true and false, usually denoted 1 and 0, respectively $\rightarrow$ Digital Logic.
- Truth Values
 - True $\rightarrow$ 1 / False $\rightarrow$ 0

3 basic gate: AND/ OR/ NOT

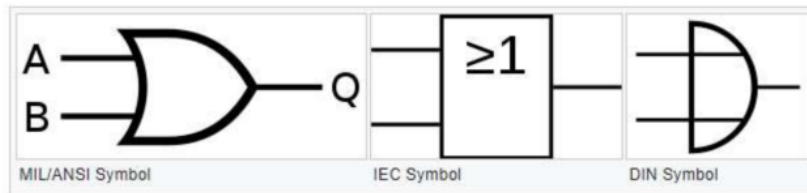
● AND Gate



MIL/ANSI Symbol IEC Symbol DIN Symbol

INPUT		OUTPUT
A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

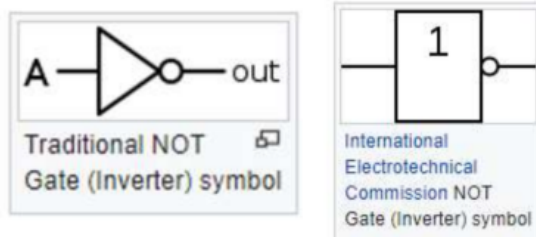
● OR Gate



MIL/ANSI Symbol IEC Symbol DIN Symbol

INPUT		OUTPUT
A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

● NOT Gate (Inverter)



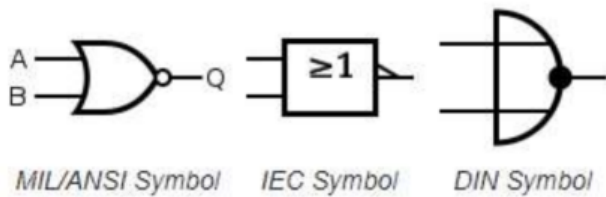
Traditional NOT Gate (Inverter) symbol

International Electrotechnical Commission NOT Gate (Inverter) symbol

INPUT	OUTPUT
A	NOT A
0	1
1	0

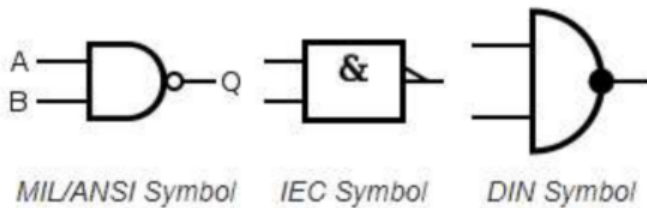
【考察：如何用 basic gate 构造 XOR gate】

● NOR Gate



INPUT		OUTPUT
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

● NAND Gate

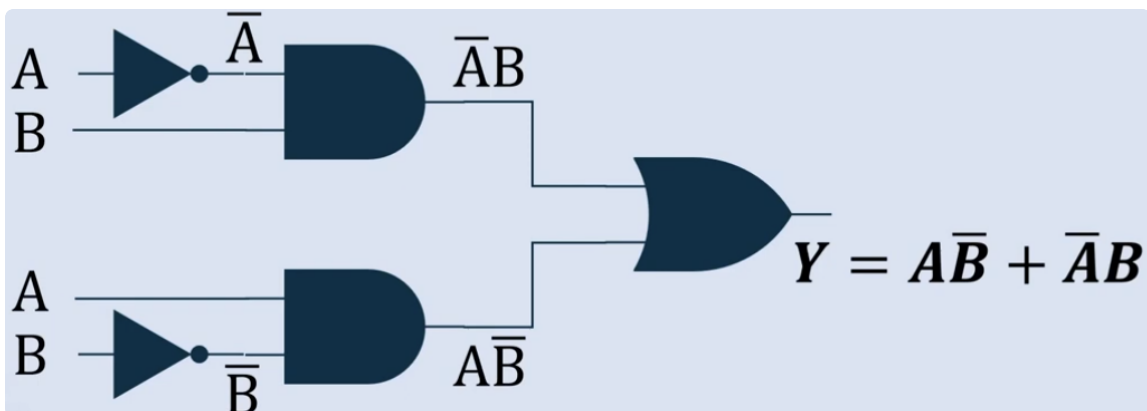


INPUT		OUTPUT
A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

● XOR (Exclusive OR) Gate



INPUT		OUTPUT
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

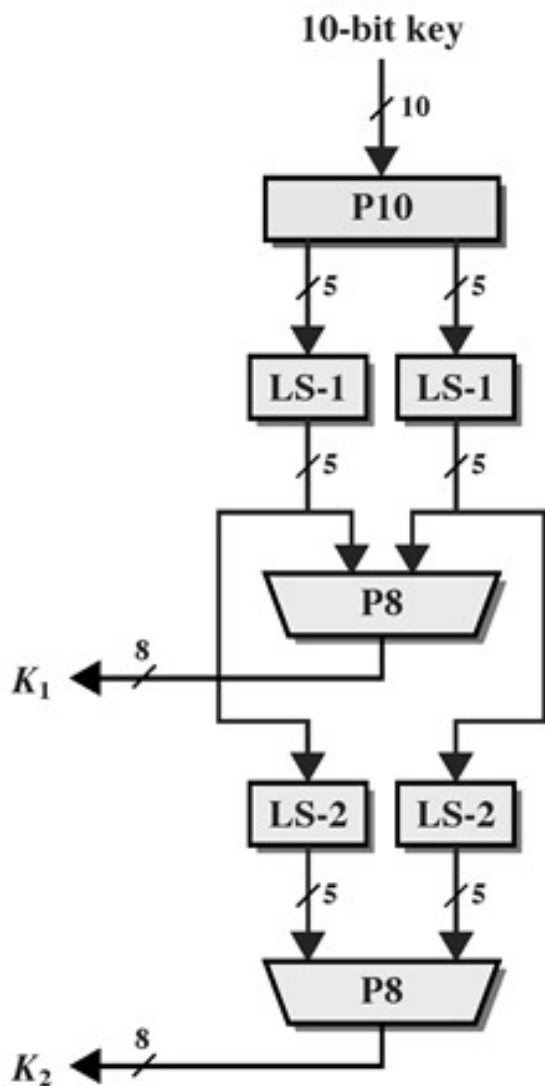


Simplified DES

- Input (plaintext) block: 8-bits 一次加密 8 位

- Output (ciphertext) block: 8-bits 这是分组加密的特点：输入 8 位 -> 输出仍然 8 位
- Key: 10-bits
- Rounds: 2
- Round keys generated using **permutations** 置换 and **left shifts** 左移
- Encryption: Initial permutation(仅重新排列位置) round function, switch halves
- Decryption: Same as encryption, except round keys used in opposite order. 解密与加密完全相同, 只是子密钥使用顺序相反 – Feistel Network 最重要性质)

1. Key Generation



Step 1: P10 (10 bit permutation) 重新排列比特位置

$P10(1,2,3,4,5,6,7,8,9,10) = \{3,5,2,7,4,10,1,9,8,6\}$

假设 原密钥: 1010000010

P10 定义: (3,5,2,7,4,10,1,9,8,6)

得到: 10000 01100

Step 2: 分成两半

1000001100 → 10000 = Left 与 01100 = Right

Step 3: LS -1 循环左移

LS - 1 就意味着 向左循环移动 1 位

则, 10000 → 00001 = Left; 01100 → 11000 = Right

拼起来 00001 11000

Step 4: P8(8 bit permutation) 从 10 位 里挑 8 位并重排

$P8(\{1,2,3,4,5,6,7,8,9,10\}) = \{6,3,7,4,8,5,10,9\}$

得到: 10100100 → K1

【重复得到 K2】

Step 5: LS -2 循环左移 2 位 (在 LS - 1 的基础上移)

得到: 00001 → 00100; 11000 → 00011

拼起来 00100 00011

Step 6: P8(8 bit permutation) 从 10 位 里挑 8 位并重排

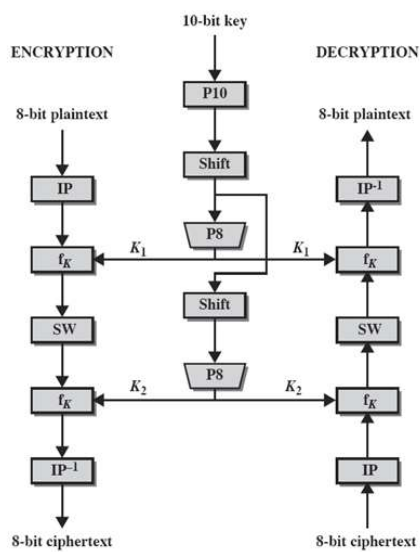
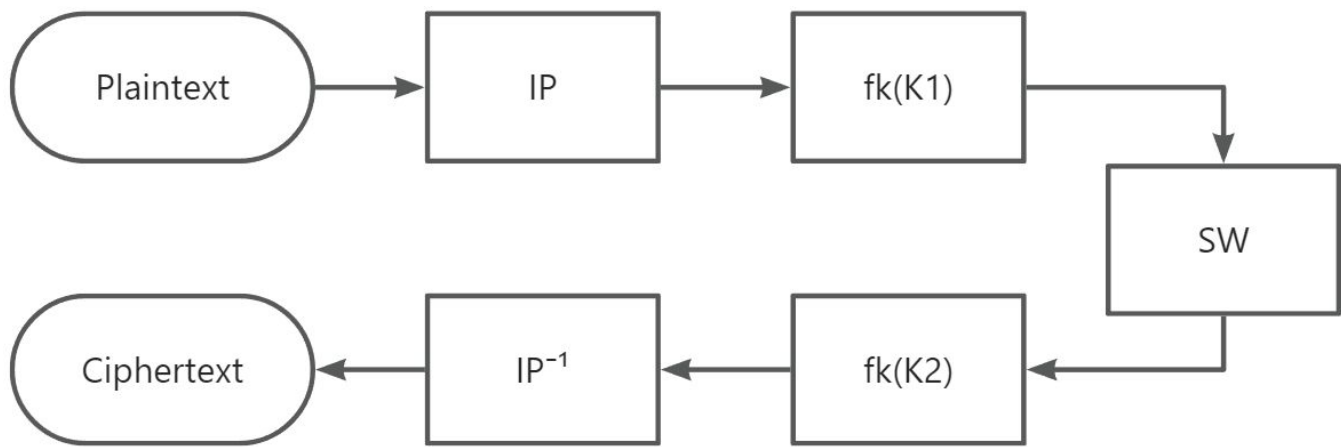
运用同样的规则: $P8(\{1,2,3,4,5,6,7,8,9,10\}) = \{6,3,7,4,8,5,10,9\}$

得到: 01000011 → K2

注意:

1. P10作用: 重新排列 10位 → 只是换位置!!!
2. P8作用: 从10位中选出8位 → 可能是乱序选择!!!

2. S-DES Algorithm



$IP = \{2, 6, 3, 1, 4, 8, 5, 7\}$

- 得到 $IP(10110010)=00111001$

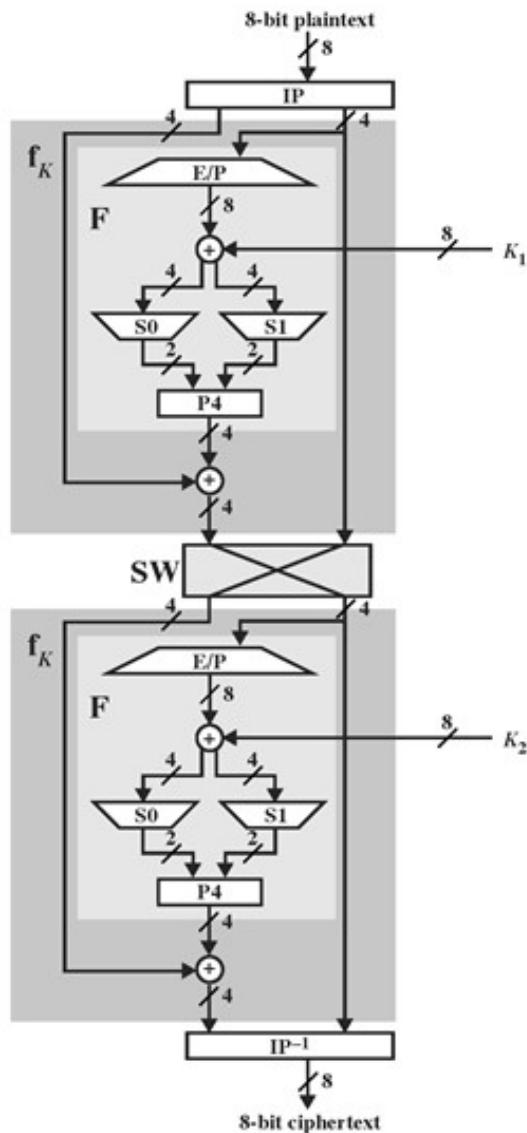
$IP^{-1}=\{4,1,3,5,7,2,8,6\}$ 逆置换: 把 IP 恢复回来

Function f_k : $f_k(L, R) = (L \oplus F(R, SK), R)$

如, 1010 0111

8 bits $\rightarrow$ 1010 = Left; 0111 = Right

右边进入 F 函数, 结果与左边异或



F 函数结构

输入：R = 4 bits

R → E/P → XOR K → S box → P4

1. E/P (Expansion Permutation)

$E/P = \{4, 1, 2, 3, 2, 3, 4, 1\}$

作用：扩展 4 bits → 8 bits

若 R = 1011, 则 $R \rightarrow E/P \rightarrow 11010111$

2. 与 K1 异或

$E/P(R) = 11010111$; $K_1 = 101010100$

得到 0111 0011

3. 分成左右各 4 位即 S0 与 S1

0111 = S0; 0011 = S1

查表：4 bits → 2 bits

4. $P4 = \{2, 4, 3, 1\}$ 重新排列 4 位

5. 输出并于 L 取异或，得到新的左半边（右边保持最初始）

6. SW 交换左右

如, 1010 0111 → 0111 1010

【重复 Round 2 到第 5 步后做 IP^{-1} 】

IP^{-1} （逆初始置换）的作用是把最开始 IP 打乱的比特顺序恢复回来；而先打乱再还原主要是为了当年 DES 硬件电路布线更方便，不是为了提高安全性。

S – Box（整个算法的非线性核心）：

- S-DES uses two S-Boxes: S0, S1
- Each S-Box is represented as a 4*4 matrix.
- The input to an S-Box is 4 bits.
- The output is 2 bits.
- The 4-bit input determines:
 - the row using the first and fourth bits; 第 1 位和第 4 位决定行；
 - the column using the second and third bits. 第 2 位和第 3 位决定列。

$$S_0 = \begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 1 & 0 & 3 & 2 \\ 3 & 2 & 1 & 0 \\ 0 & 2 & 1 & 3 \\ 3 & 1 & 3 & 2 \end{bmatrix} \end{matrix} \quad S_1 = \begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 3 \\ 3 & 0 & 1 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix} \end{matrix}$$

若输入为 b1b2b3b4

则：行号由b1b4决定；列号由b2b3决定

查表得到一个十进制数，再将其转换成 2 位二进制输出。

若，输入为 1011

则 b1b4 = 11 → 3, b2b3 = 01 → 1, $S_0[3][1] = 1 \rightarrow 01$; S1同理

名称	作用
P10	对 10 位原始密钥进行排列
P8	从移位后的 10 位中选择并排列 8 位
IP	对 8 位明文/密文做初始置换
IP^{-1}	撤销 IP 的位置变化
E/P	将 4 位扩展并排列为 8 位
P4	对 S-Box 产生的 4 位结果重新排列

3. Example

在 S-DES/DES 中，IP、S-Box、各种 P 置换的位置表都是算法设计者事先规定好的固定参数，即算法规则公开且固定，密钥秘密且可变

■ Given

- plaintext 1001 0101
- key 10011 01100

■ Key generation

- P10 → 01011 01010
- LS-1 → 10110 10100
 - P8 → 1101 1000 (k1)
- LS-2 → 11010 10010
 - P8 → 1001 0001 (k2)

Step 1: P10 (10 bit permutation) 重新排列比特位置 并分左右

$P10(1,2,3,4,5,6,7,8,9,10) = \{3,5,2,7,4,10,1,9,8,6\}$

10011 01100 → 01011 = Left 01010 = Right

Step 2: LS -1 循环左移

10110 10100

Step 3: P8(8 bit permutation) 从 10 位 里挑 8 位并重排

$P8(\{1,2,3,4,5,6,7,8,9,10\}) = \{6,3,7,4,8,5,10,9\}$

1101 1000 (K1)

Step 4: LS -2 循环左移 2 位 (在 LS - 1 的基础上移)

11010 10010

Step 5: P8(8 bit permutation) 从 10 位 里挑 8 位并重排

运用同样的规则: $P8(\{1,2,3,4,5,6,7,8,9,10\}) = \{6,3,7,4,8,5,10,9\}$

得到: 1001 0001 (K2)

● Encrypt: 1001 0101

■ IP → 0101 1100 L: 0101 R: 1100

■ $f_k(L, R) = (L \oplus F(R, S_{K1}), R)$

- EP → 0110 1001
- $\oplus K_1 \rightarrow 1011 0001$
- $S_0 \rightarrow \text{row } 11, \text{col } 01 \rightarrow 01$
- $S_1 \rightarrow \text{row } 01, \text{col } 00 \rightarrow 10$
- P4 → 1010
- $0101 \oplus 1010 \rightarrow 1111$

■ SW → 1100 1111

Ciphertext: 1101 1111

L: 1100 R: 1111

■ $f_k(L, R) = (L \oplus F(R, S_{K2}), R)$

- EP → 1111 1111
- $\oplus K_2 \rightarrow 0110 1110$
- $S_0 \rightarrow \text{row } 00, \text{col } 11 \rightarrow 10$
- $S_1 \rightarrow \text{row } 10, \text{col } 11 \rightarrow 00$
- P4 → 0001
- $1100 \oplus 0001 \rightarrow 1101$
- $IP^{-1} \rightarrow 1101 1111$

● Decrypt: 1101 1111

■ IP → 1101 1111 L: 1101 R: 1111

■ $f_k(L, R) = (L \oplus F(R, S_{K2}), R)$

- EP → 1111 1111
- $\oplus K_2 \rightarrow 0110 1110$
- $S_0 \rightarrow \text{row } 00, \text{col } 11 \rightarrow 10$
- $S_1 \rightarrow \text{row } 10, \text{col } 11 \rightarrow 00$
- P4 → 0001
- $1101 \oplus 0001 \rightarrow 1100$

■ SW → 1111 1100

Plaintext: 1001 0101

L: 1111 R: 1100

■ $f_k(L, R) = (L \oplus F(R, S_{K1}), R)$

- EP → 0110 1001
- $\oplus K_1 \rightarrow 1011 0001$
- $S_0 \rightarrow \text{row } 11, \text{col } 01 \rightarrow 01$
- $S_1 \rightarrow \text{row } 01, \text{col } 00 \rightarrow 10$
- P4 → 1010
- $1111 \oplus 1010 \rightarrow 0101$
- $IP^{-1} \rightarrow 1001 0101$

IP 得到 8 bits, 分左右, 右边送入 f_k , 得到的结果与左边初始值取异或后, 作为左边的结果;
右边直接用最初值

加密:

Step 1: IP = {2, 6, 3, 1, 4, 8, 5, 7} 并分左右, 并送 right 进 f_k

1001 0101 → 0101 = Left; 1100 = Right

Step 2: E/P (Expansion Permutation) $E/P=\{4,1,2,3,2,3,4,1\}$

1100 → 0110 1001

Step 3: 与 K1 异或, 并分左右

$E/P(R) = 0110\ 1001$; 1101 1000 (K1) → 1011 = Left; 0001 = Right

Step 4: 查表

$$S0 = \begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 1 & 0 & 3 & 2 \\ 3 & 2 & 1 & 0 \\ 0 & 2 & 1 & 3 \\ 3 & 1 & 3 & 2 \end{bmatrix} \end{matrix} \quad S1 = \begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 3 \\ 3 & 0 & 1 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix} \end{matrix}$$

Left = 1011 → [3,1] → 1 → 01; Right = 1000 → [1,0] → 2 → 10

Step 5: $P4=\{2,4,3,1\}$ 重新排列 4 位, 得到 0110 → 1010

Step 6: 输出并于 L 取异或, 得到新的左半边 (右边保持原样) 0101 与 1010 取异或 → 1111

- 输出 1111 1100

Step 7: SW 交换左右 1111 1100 → 1100 1111

【重复 Round 2】

Step 2.2: E/P (Expansion Permutation) $E/P=\{4,1,2,3,2,3,4,1\}$

1111 → 1111 1111

Step 3.2: 与 K2 异或, 并分左右

$E/P(R) = 1111\ 1111$; 1001 0001 (K2) → 0110 = Left; 1110 = Right

Step 4.2: 查表

$$S0 = \begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 1 & 0 & 3 & 2 \\ 3 & 2 & 1 & 0 \\ 0 & 2 & 1 & 3 \\ 3 & 1 & 3 & 2 \end{bmatrix} \end{matrix} \quad S1 = \begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 1 & 2 & 3 \\ 2 & 0 & 1 & 3 \\ 3 & 0 & 1 & 0 \\ 2 & 1 & 0 & 3 \end{bmatrix} \end{matrix}$$

Left = 0110 → [0,3] → 2 → 10; Right = 1110 → [2,3] → 0 → 00

Step 5.2: $P4=\{2,4,3,1\}$ 重新排列 4 位, 得到 1000 → 0001

Step 6.2: 输出并于 L 取异或, 得到新的左半边 (右边保持原样) 0001 与 1100 取异或 → 1101

- 输出 1101 1111

Step 7: $IP^{-1}=\{4,1,3,5,7,2,8,6\}$ 得到 1101 1111

解密：在 S-DES 中，解密和加密使用相同的整体结构，只把 K_1 、 K_2 的使用顺序倒过来

对，在 S-DES 中，解密和加密使用相同的整体结构，只把 K_1 、 K_2 的使用顺序倒过来。

加密：

$$P \xrightarrow{IP} f_{K_1} \xrightarrow{SW} f_{K_2} \xrightarrow{IP^{-1}} C$$

解密：

$$C \xrightarrow{IP} f_{K_2} \xrightarrow{SW} f_{K_1} \xrightarrow{IP^{-1}} P$$

即：

- 加密：先 K_1 ，后 K_2
- 解密：先 K_2 ，后 K_1
- IP 、 SW 、 IP^{-1} 、S-Box 等规则不变

之所以可行，是因为 S-DES 使用 Feistel 结构，并且异或具有“同一个值异或两次会被抵消”的性质：

$$A \oplus B \oplus B = A$$

注意，不是简单地把加密的所有步骤从后往前执行；准确说是保持算法框架不变，只反转轮密钥的顺序。

4. Summary

● Simplified DES expressed as functions:

$$CT = IP^{-1} \circ f_{K_2} \circ SW \circ f_{K_1} \circ IP(PT)$$

$$PT = IP^{-1} \circ f_{K_1} \circ SW \circ f_{K_2} \circ IP(CT)$$

● Security of Simplified DES:

- 10-bit key, 1024 keys: brute force easy.
- If know plaintext and corresponding ciphertext, can we determine key? → **Very hard**